

# Capítulo 1

## Marco teórico

El propósito de este capítulo es reunir los fundamentos que sostienen el detector construido en el caso práctico: cómo se convierte un texto en números, qué modelos clasifican esos números, cómo se validan sin engañarse y con qué métricas se comparan. Cada concepto aparece una sola vez y en su forma general; su aplicación concreta, con los datos, las variables y las cifras del problema, se reserva para el capítulo siguiente.

### 1.1. Text mining

El dato de partida es texto, pero los clasificadores que usaremos operan sobre vectores de números reales. La minería de textos (*text mining*) [1] es el conjunto de técnicas que salva esa distancia: transforma un texto en una representación numérica sobre la que un modelo pueda calcular. Reunimos aquí las tres piezas que necesita el detector: trocear el texto en unidades (tokenización), pesar cada unidad según lo informativa que sea (ponderación TF-IDF) y, con esa representación, medir cuánto se parecen dos textos (medidas de similitud). El vocabulario de esta sección es el que emplea el capítulo práctico para construir las variables a partir de cada respuesta y su fuente.

**Tokenización y normalización.** El primer paso rompe el texto en unidades mínimas llamadas *tokens*. Una tokenización simple pasa el texto a minúsculas y extrae las secuencias alfanuméricas maximales, de modo que “WiFi” y “wifi” cuenten como la misma palabra y los signos de puntuación se descarten. La frase “El hotel tiene WiFi gratis.” produce entonces los cinco tokens ⟨el, hotel, tiene, wifi, gratis⟩. Sobre esa secuencia se definen los *n-gramas*: grupos de  $n$  tokens consecutivos. Un unigrama es una palabra suelta; un bigrama, dos seguidas (“wifi gratis”); un trigramas, tres. Los  $n$ -gramas retienen combinaciones que la palabra aislada pierde: “no funciona” dice lo contrario que “funciona”, y solo el bigrama lo captura. Esa ganancia tiene un precio, cada nivel de  $n$ -grama multiplica el tamaño del vocabulario, así que en la práctica se limita a unigramas y bigramas.

Dos técnicas reducen las variantes de una palabra a una forma común, para no tratar “jugar” y “jugando” como términos distintos. El *stemming* recorta la palabra a su raíz mediante reglas (“jugando”, “jugador” → “jug”); la *lematización* la lleva a su forma de diccionario usando su categoría gramatical (“jugando” → “jugar”), lo que es más preciso y más costoso. En este trabajo la tokenización se mantiene simple, sin stemming ni lematización, porque la señal que buscamos vive en el solape literal de palabras entre respuesta y fuente, y normalizar las formas la difuminaría.

**Bolsa de palabras y TF-IDF.** La representación más básica de un texto es la *bolsa de palabras* [18]: se fija un vocabulario de  $V$  términos, uno por cada palabra distinta de la colección, y cada texto se convierte en un vector de  $\mathbb{R}^V$  que cuenta cuántas veces aparece cada término, sin atender al orden en que salen. Con los documentos “free wifi hotel”, “wifi hotel breakfast” y “cheap hotel”, el vocabulario tiene cinco términos y cada documento es una fila de la matriz de frecuencias:

|                              | free | wifi | hotel | breakfast | cheap |
|------------------------------|------|------|-------|-----------|-------|
| $d_1$ : free wifi hotel      | 1    | 1    | 1     | 0         | 0     |
| $d_2$ : wifi hotel breakfast | 0    | 1    | 1     | 1         | 0     |
| $d_3$ : cheap hotel          | 0    | 0    | 1     | 0         | 1     |

El defecto de esta representación es dar el mismo peso a una palabra vacía muy repetida (“de”, “el”) que a un término informativo raro. El esquema **TF-IDF** (*term frequency-inverse document frequency*) corrige ese sesgo ponderando cada término por su rareza en la colección [21]: pesa mucho lo específico y poco lo frecuente. Sea una colección de  $N$  documentos; para un término  $t$ , denotamos por  $\text{tf}(t, d)$  el número de veces que  $t$  aparece en el documento  $d$  y por  $\text{df}(t)$  el número de documentos que contienen  $t$ . El peso de  $t$  en  $d$  es el producto de dos factores,

$$w_{t,d} = \underbrace{\text{tf}(t, d)}_{\text{frecuencia en el documento}} \cdot \underbrace{\log \frac{N}{\text{df}(t)}}_{\text{rareza en la colección}}. \quad (1.1)$$

El primer factor, la frecuencia de término, sube con las veces que  $t$  aparece en el documento; el segundo, la frecuencia inversa de documento, lo penaliza si además aparece en muchos documentos. La intuición del segundo factor es una medida de sorpresa: un término presente en casi todos los documentos tiene  $\text{df}(t) \approx N$ , su logaritmo tiende a cero y su peso se anula, porque verlo no informa; uno raro tiene  $\text{df}(t)$  pequeño y su logaritmo crece. Sobre el ejemplo anterior, con  $N = 3$  y tomando logaritmo natural, el factor de rareza es:

| término                | $\text{df}(t)$ | $\ln(N/\text{df}(t))$   |
|------------------------|----------------|-------------------------|
| hotel                  | 3              | $\ln(3/3) = 0$          |
| wifi                   | 2              | $\ln(3/2) \approx 0,41$ |
| free, breakfast, cheap | 1              | $\ln(3/1) \approx 1,10$ |

La palabra “hotel”, presente en los tres documentos, recibe peso cero: es común y no distingue nada. Las que solo salen en uno pesan lo máximo, porque son las que lo caracterizan frente al resto. Así, la fila de  $d_1$  en TF-IDF deja de ser  $(1, 1, 1, 0, 0)$  y pasa a  $(1,10, 0,41, 0, 0, 0)$ : “free” manda, “hotel” desaparece.

**Similitud entre dos textos.** La señal central del detector es cuánto se apoya una respuesta en su fuente: una respuesta cuyas palabras están casi todas respaldadas por la fuente es fiable, mientras que una que introduce términos ajenos a ella levanta la sospecha de invención. Para cuantificarlo representamos cada texto por su conjunto de palabras únicas.

**Definición 1** (Conjuntos de palabras de la respuesta y la fuente). Dada una respuesta y su fuente, sea  $R$  el conjunto de palabras únicas de la respuesta y  $F$  el conjunto de palabras únicas de la fuente.  $|R|$  y  $|F|$  son sus cardinales,  $R \cap F$  las palabras comunes y  $R \cup F$  el total de palabras distintas entre las dos.

Sobre  $R$  y  $F$  se definen tres medidas de solape con lecturas distintas:

$$\text{containment} = \frac{|R \cap F|}{|R|}, \quad \text{Jaccard} = \frac{|R \cap F|}{|R \cup F|}, \quad \cos(u, v) = \frac{\langle u, v \rangle}{\|u\| \|v\|}. \quad (1.2)$$

El **containment** es la fracción de palabras de la respuesta que también están en la fuente; mira solo hacia la respuesta, así que no penaliza que la fuente sea larga. El **índice de Jaccard** es el solape simétrico, normalizado por la unión, y sí baja cuando los textos difieren mucho en tamaño. El **coseno TF-IDF** no trabaja sobre conjuntos, sino sobre los vectores TF-IDF  $u$  y  $v$  de ambos textos, y mide el coseno del ángulo que forman (Figura 1.1) a partir de su producto escalar  $\langle u, v \rangle$  y sus normas: vale 1 cuando apuntan en la misma dirección, 0 cuando son perpendiculares, y pondera cada palabra por su rareza, de modo que coincidir en un término raro cuenta más que coincidir en uno común. Con  $R = \{\text{free, wifi, indian}\}$  y  $F = \{\text{wifi, indian, vegan}\}$  la intersección tiene dos palabras y la unión cuatro, luego el containment es  $|R \cap F|/|R| = 2/3 \approx 0,67$  y el Jaccard  $|R \cap F|/|R \cup F| = 2/4 = 0,50$ . El containment queda por encima porque ignora la palabra “vegan”, presente solo en la fuente; el Jaccard sí la cuenta en el denominador de la unión y por eso baja.

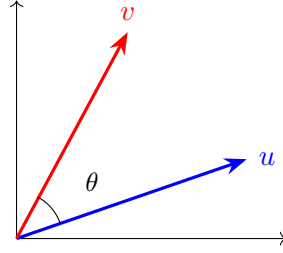


Figura 1.1: El coseno mide la similitud por el ángulo  $\theta$  entre los vectores de dos textos, no por su longitud: dos textos con las mismas proporciones de palabras apuntan en la misma dirección.

El containment cae cuando la respuesta introduce palabras nuevas, ausentes de la fuente, que es la huella de una invención léxica: es la medida que más nos interesa, porque captura directamente el anclaje de la respuesta en su fuente.

**Representación semántica.** El solape de palabras no capta sinónimos ni paráfrasis: “coche” y “automóvil” cuentan como términos sin relación, aunque signifiquen lo mismo. Una representación que sí los acerca, sin recurrir a redes neuronales, es el *análisis semántico latente* (LSA) [10]. Parte de la matriz TF-IDF  $M \in \mathbb{R}^{N \times V}$ , con una fila por documento y una columna por término, y le aplica una descomposición en valores singulares truncada a  $k$  componentes,

$$M \approx U_k \Sigma_k V_k^\top, \quad (1.3)$$

donde  $\Sigma_k$  es la matriz diagonal de los  $k$  mayores valores singulares y las columnas de  $U_k$  y  $V_k$  son los vectores singulares asociados. Esos  $k$  ejes concentran la mayor variación de la matriz; proyectar los textos sobre ellos,  $M V_k$ , los lleva a un espacio de  $k$  dimensiones donde los términos que suelen aparecer juntos quedan cerca, de modo que dos textos que comparten temática resultan próximos aunque no compartan las mismas palabras. Con  $V$  grande, calcular la factorización exacta es caro, y en la práctica se usa un algoritmo aleatorizado que la aproxima con un coste mucho menor [16]. Es la única representación semántica que usamos, coherente con un enfoque clásico y barato.

## 1.2. Metodología

Un proyecto de modelización analítica sigue un flujo ordenado de fases. Adoptamos la metodología **SEMMA** [2], propuesta por SAS, que organiza el proceso de minería de datos en cinco fases (Figura 1.2): exploración, muestreo, modificación, modelización y valoración. Es una organización lógica de las tareas del análisis, hermana de la metodología CRISP-DM [24], y estructura tanto este capítulo como su aplicación en el práctico. Cada fase cumple un papel:

- **Exploración** (*Explore*): describir los datos, sus distribuciones, la prevalencia de cada clase y las relaciones entre variables.
- **Muestreo** (*Sample*): fijar los conjuntos de entrenamiento, validación y test, y equilibrar las clases si el reparto es muy desigual.
- **Modificación** (*Modify*): depurar y preparar las variables (valores ausentes, atípicos) y seleccionar las informativas.
- **Modelización** (*Model*): ajustar los modelos y sus hiperparámetros.
- **Valoración** (*Assess*): medir el rendimiento de los modelos ajustados con métricas.

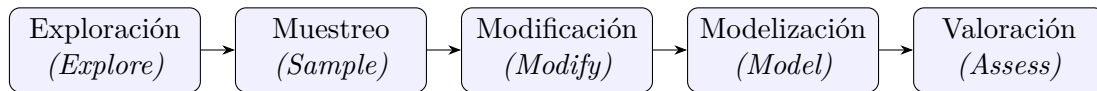


Figura 1.2: Las cinco fases de la metodología SEMMA, en el orden en que se aplican al problema.

Las cinco fases se desarrollan a continuación.

### 1.2.1. Análisis exploratorio

Antes de modelar hay que mirar los datos. El **análisis descriptivo** caracteriza cada variable y la etiqueta: su distribución, su rango, sus valores atípicos y las relaciones que guardan entre sí y con la clase. De ese examen salen las decisiones de las fases siguientes y afloran los problemas que condicionan el resto del trabajo. El más determinante en clasificación es el **desbalance** entre clases. La *prevalencia* de una clase es la fracción de ejemplos que le pertenecen; cuando una clase es mucho menos frecuente que la otra, el aprendizaje se sesga hacia la mayoritaria, y un modelo que se limite a predecirla siempre acierta en apariencia sin detectar un solo caso de la minoritaria. Si una clase supone el 90 % de los ejemplos, contestar siempre “mayoritaria” logra un 90 % de aciertos y cero utilidad. Reconocer el desbalance a tiempo orienta las tres decisiones posteriores: el muestreo con que se equilibra la base, el umbral con que se decide y las métricas con que se juzga, las tres cuestiones que retoman las subsecciones siguientes.

### 1.2.2. Muestreo

Cuando el reparto entre clases es muy desigual, el entrenamiento tiende a sesgarse hacia la mayoritaria. El **remuestreo** reequilibra la base antes de entrenar de dos formas: el *sobremuestreo* (*oversampling*) replica o sintetiza ejemplos de la clase minoritaria (SMOTE los interpola entre vecinos cercanos [6]) y el *submuestreo* (*undersampling*) descarta ejemplos de la mayoritaria. El remuestreo no es la única palanca frente al desbalance: el umbral de decisión, que fijamos en la validación, ajusta el compromiso entre clases sin tocar los datos.

### 1.2.3. Manipulación de datos

La fase de **modificación** prepara las variables antes de modelar. Depura los datos (imputa los valores ausentes, o *missing*, con un valor plausible y acota los atípicos, o *outliers*, que distorsionan sobre todo a los modelos lineales) y reduce la dimensión quedándose con las variables informativas. La construcción de las variables a partir del texto, la ingeniería de características, se describe en el capítulo práctico; aquí tratamos la teoría de la selección de variables.

**Selección de características.** Con muchas variables conviene quedarse con las informativas y descartar las redundantes, tanto para mejorar el modelo como para entenderlo [14]. Hay dos familias. Los métodos de **filtro** puntúan cada variable por una medida estadística de su relación con la etiqueta, al margen del modelo; son rápidos, pero ignoran las interacciones y la redundancia entre variables, así que pueden quedarse con dos variables casi idénticas. Los métodos **envolventes** (*wrapper*) usan el propio modelo como juez: prueban subconjuntos y se quedan con el que mejor valida, a mayor coste. La **eliminación recursiva de características** (RFE) es el envoltorio que usamos [15]: entrena con todas las variables, elimina la menos importante, reentrena y repite, lo que produce un orden de importancia. El número final de variables se fija por el *método del codo*, el punto de la curva de rendimiento a partir del cual añadir variables ya no aporta.

### 1.2.4. Modelización

Entrenar un clasificador es ajustar sus *parámetros* para que reproduzca las etiquetas del entrenamiento, minimizando una función de pérdida. Los modelos concretos que entrenamos se desarrollan más adelante, en la sección de modelos; queda antes un paso común a todos: fijar sus *hiperparámetros*, las opciones que no se aprenden de los datos (la profundidad de un árbol, el número de vecinos) y que hay que decidir de antemano.

**Validación cruzada.** Medir el rendimiento sobre los mismos datos con que se entrena infla el resultado, porque el modelo ya los conoce. Reservar una parte para test lo evita, pero entonces el resultado depende de qué ejemplos tocaron en esa partición. La **validación cruzada** de  $k$  iteraciones elimina esa dependencia [23]: parte el entrenamiento en  $k$  bloques, entrena con  $k - 1$  y valida con el restante, y repite rotando el bloque de validación (Figura 1.3); el rendimiento final es el promedio de las  $k$  rondas, que ya no depende de una partición afortunada.

Esta partición debe respetar la estructura de los datos. Cuando varios ejemplos comparten una misma unidad, aquí un mismo documento fuente del que salen varias respuestas, esa unidad tiene que caer entera en un solo bloque. Si no, el modelo ve en validación una respuesta casi idéntica a otra que tuvo en entrenamiento y el resultado se infla por esa filtración. La variante que lo garantiza es el **GroupKFold**, que agrupa por la unidad e impide que un documento aparezca a la vez en entrenamiento y validación.

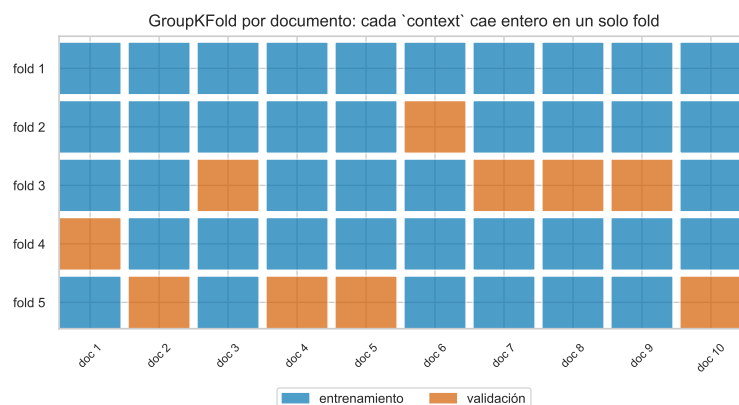


Figura 1.3: GroupKFold por documento: cada fuente cae entera en un único bloque, de modo que ninguna aparece a la vez en entrenamiento (azul) y validación (naranja).

**Búsqueda en rejilla.** Casi todos los modelos tienen hiperparámetros, y distintas combinaciones dan resultados distintos: no es lo mismo tomar los 3 vecinos más cercanos con distancia euclídea que los 11 con distancia de Manhattan. La **búsqueda en rejilla** (*grid search*) los ajusta probando un conjunto de combinaciones, evaluando cada una por validación cruzada y conservando la mejor, sin tocar el test (Figura 1.4). Su coste crece con el producto de los valores de cada hiperparámetro por las iteraciones de la validación: optimizar tres hiperparámetros con tres valores cada uno y validación cruzada de cuatro iteraciones ya son  $3 \cdot 3 \cdot 3 \cdot 4 = 108$  ajustes. Para modelos clásicos ese coste es asumible en un ordenador corriente.

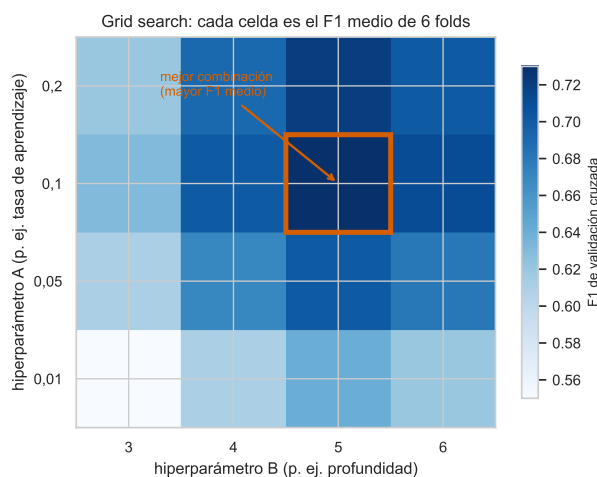


Figura 1.4: Búsqueda en rejilla: cada celda es el rendimiento medio de una combinación de hiperparámetros en validación cruzada; se conserva la mejor.

### 1.2.5. Valoración

La fase de **valoración** mide el rendimiento de los modelos ajustados. Necesita un lenguaje de métricas que cuantifique el acierto y distinga los tipos de error.

Evaluar un clasificador binario empieza por contar sus aciertos y errores. Tomando como clase positiva la que interesa detectar (aquí, la alucinación), cada predicción cae en una de cuatro casillas: los verdaderos positivos (VP) y verdaderos negativos (VN) son aciertos; los

falsos positivos (FP, error de tipo I) y falsos negativos (FN, error de tipo II) son los dos tipos de fallo. Su recuento se organiza en la *matriz de confusión* (Figura 1.5).

|          |         | Predicción                                    |                                      |
|----------|---------|-----------------------------------------------|--------------------------------------|
|          |         | limpia                                        | alucina                              |
| Realidad | alucina | VP<br>(alucina bien detectada)                | FP<br>(falsa alarma)<br>Error tipo I |
|          | limpia  | FN<br>(alucinación no vista)<br>Error tipo II | VN<br>(limpia bien detectada)        |

Figura 1.5: Matriz de confusión. El falso negativo (tipo II) es un positivo real que se predice negativo; el falso positivo (tipo I), un negativo que se predice positivo.

Los dos errores rara vez cuestan lo mismo: en un diagnóstico médico, un falso negativo deja sin tratar a un enfermo, mientras que un falso positivo trata a un sano. En nuestro caso el falso negativo, una alucinación que se cuela, es el error caro, y las métricas siguientes permiten primar su detección.

Sobre los cuatro conteos se definen las dos medidas básicas. La **precisión** indica cuántas de las predicciones positivas fueron correctas, es decir, cómo de fiable es el modelo cuando marca algo como positivo. El **recall** (también exhaustividad o sensibilidad) indica cuántos de los positivos reales llegó a detectar:

$$\text{precisión} = \frac{VP}{VP + FP}, \quad \text{recall} = \frac{VP}{VP + FN}. \quad (1.4)$$

Precisión y recall se mueven en sentidos opuestos, y el equilibrio lo fija el umbral de decisión. Si el modelo solo predice positivo cuando está muy seguro, acierta casi siempre que lo hace (precisión alta), pero deja escapar muchos positivos dudosos (recall bajo). Si predice positivo a la mínima, no se le escapa casi ninguno (recall alto) a costa de muchas falsas alarmas (precisión baja). En el extremo, un modelo que marca todo como positivo tiene recall 1 y precisión igual a la simple proporción de positivos.

Resumir precisión y recall en un solo número exige combinarlos de forma que ninguno pueda salvarse a costa del otro. La media aritmética no sirve: valdría 0,5 para un modelo con precisión 1 y recall 0, que no detecta nada. La media armónica sí penaliza ese desequilibrio.

**Definición 2** (Media armónica). La *media armónica* de dos números positivos  $a$  y  $b$  es

$$H(a, b) = \frac{2}{\frac{1}{a} + \frac{1}{b}} = \frac{2ab}{a + b}.$$

Está siempre dominada por la media aritmética, se acerca al menor de los dos valores y tiende a cero en cuanto uno de ellos lo hace.

El valor **F1** es la media armónica de precisión y recall, y solo es alto cuando los dos lo son:

$$F_1 = H(\text{precisión}, \text{recall}) = 2 \frac{\text{precisión} \cdot \text{recall}}{\text{precisión} + \text{recall}}. \quad (1.5)$$

El F1 supone que precisión y recall importan igual, lo que no siempre ocurre. Cuando detectar positivos pesa más que evitar falsas alarmas, se usa su generalización  $F_\beta$ , que valora el recall  $\beta^2$  veces más que la precisión:

$$F_\beta = (1 + \beta^2) \frac{\text{precisión} \cdot \text{recall}}{\beta^2 \text{precisión} + \text{recall}}. \quad (1.6)$$

El caso  $\beta = 2$ , denotado  $F_2$ , pesa el recall cuatro veces más que la precisión, y conviene cuando el falso negativo es el error caro, como en la detección de alucinaciones.

Una medida más intuitiva, la **accuracy** o fracción de aciertos,  $\frac{VP+VN}{VP+VN+FP+FN}$ , engaña cuando una clase domina: si el 90 % de los ejemplos son negativos, predecir siempre negativo ya da un 90 % de accuracy sin detectar un solo positivo. La **balanced accuracy** corrige ese espejismo promediando el recall de cada clase. Un ejemplo lo fija: sobre 300 respuestas con 100 alucinaciones reales, un detector caza 70 ( $VP = 70$ ,  $FN = 30$ ) a costa de 20 falsas alarmas ( $FP = 20$ ,  $VN = 180$ ). Su precisión es  $70/90 \approx 0,78$ , el recall  $70/100 = 0,70$  y el  $F_1 \approx 0,74$ ; la accuracy sube a  $(70+180)/300 \approx 0,83$  porque acierta casi todos los negativos, que son mayoría, mientras que la balanced accuracy,  $\frac{1}{2}(0,70 + 0,90) = 0,80$ , rebaja esa cifra optimista.

Todas las métricas anteriores dependen del umbral con que la probabilidad se convierte en decisión. Dos curvas recorren todos los umbrales de golpe. La **curva ROC** (Figura 1.6, izquierda) contrapone la tasa de verdaderos positivos (el recall) frente a la tasa de falsos positivos al variar el umbral [11]; su área bajo la curva es el **AUC**. El AUC admite dos lecturas equivalentes, una geométrica y otra probabilística, que el teorema siguiente enuncia.

**Teorema 1** (Área bajo la curva ROC). Sea  $s(x)$  la puntuación que el clasificador asigna a un ejemplo  $x$ , y sean  $X^+$  y  $X^-$  un positivo y un negativo tomados al azar de forma independiente. El área bajo la curva ROC coincide con la probabilidad de que el positivo puntúe por encima del negativo:

$$\text{AUC} = \int_0^1 \text{TVP}(\text{TFP}) d(\text{TFP}) = \mathbb{P}(s(X^+) > s(X^-)),$$

con el convenio de contar como  $\frac{1}{2}$  los empates  $s(X^+) = s(X^-)$ .

La primera igualdad es la definición geométrica, el área bajo la curva; la segunda es su lectura probabilística [17]. El AUC tiene dos propiedades que lo hacen una medida sólida para comparar modelos: es invariante a la escala del score, pues solo importa el orden de las predicciones, y es invariante al umbral, pues los resume todos. La **curva precisión–recall** (Figura 1.6, derecha), en cambio, sí refleja el desbalance de clases, porque la precisión depende de la proporción de positivos; un modelo bueno mantiene la precisión alta aun a recall alto, y su curva se acerca a la esquina superior derecha. Por eso la miramos además del AUC cuando la clase positiva es minoritaria.



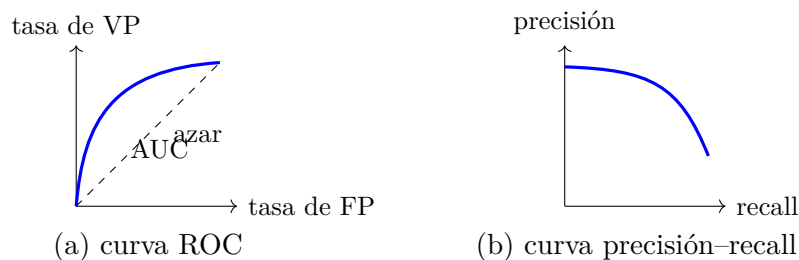


Figura 1.6: Las dos curvas que recorren todos los umbrales. La ROC resume la separación en el AUC; la precisión–recall muestra a partir de qué recall se degrada la precisión.

Elegido un modelo, queda fijar el umbral  $t$  que decide positivo cuando el score supera  $t$ . El **índice de Youden** elige el que maximiza  $J = \text{sensibilidad} + \text{especificidad} - 1$ , esto es, el punto de la curva ROC más alejado de la diagonal, donde mejor se separan las dos clases [26].

**Umbral de decisión.** El clasificador devuelve una probabilidad, y convertirla en una decisión exige fijar un **umbral** de corte. El índice de Youden da un umbral que separa bien las dos clases en promedio, pero no es la única opción: bajar el umbral gana recall a costa de precisión, y subirlo hace lo contrario. Cuando la prevalencia cambia entre grupos, un umbral distinto por grupo aprovecha esa diferencia. El umbral es así la segunda palanca frente al desbalance, junto al remuestreo.

## 1.3. Fundamentos de los modelos utilizados

Esta sección reúne los fundamentos de los cinco clasificadores del caso práctico. Empezamos por los conceptos generales del aprendizaje automático y desarrollamos después cada modelo con su intuición, su formulación y sus ventajas y límites.

### 1.3.1. Introducción al aprendizaje automático

Un modelo de aprendizaje automático es un programa que aprende a reconocer patrones a partir de ejemplos [19]. En lugar de escribir a mano las reglas de decisión, algo inviable cuando el fenómeno es complejo, se le muestra un conjunto de datos ya resueltos y ajusta sus parámetros para reproducirlos; hecho esto, sabe responder ante datos nuevos que no había visto. Según lo que se conozca de los ejemplos, se distinguen dos grandes tipos de aprendizaje:

- **Supervisado.** Los ejemplos vienen etiquetados con la respuesta correcta y el modelo aprende a predecirla. Es el que usamos aquí.
- **No supervisado.** No hay etiqueta; el modelo agrupa los datos por semejanza. Queda fuera de este trabajo.

Dentro del aprendizaje supervisado, la naturaleza de la respuesta separa dos tareas:

- **Regresión.** La respuesta es un número continuo, como el tiempo de un trayecto o el precio de una vivienda.
- **Clasificación.** La respuesta es una categoría de un conjunto finito; cuando solo hay dos valores se habla de **clasificación binaria**, como decidir si un correo es spam o si un paciente está enfermo.

La Figura 1.7 contrasta ambas. Nuestro problema es de clasificación binaria: decidir si una respuesta contiene una alucinación o no.

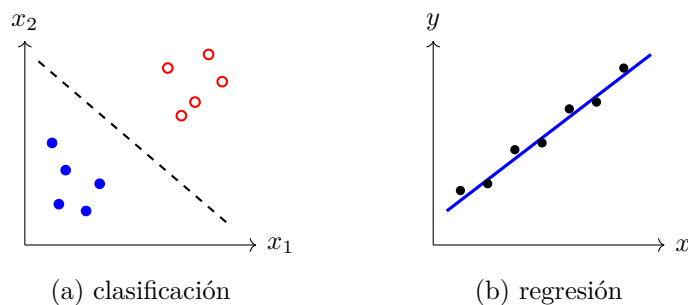


Figura 1.7: En clasificación la salida es una categoría y el modelo busca una frontera entre clases; en regresión la salida es continua y el modelo busca la tendencia.

Antes de fijar el objetivo del aprendizaje conviene nombrar los objetos con que trabaja. Cada ejemplo de la base de datos se describe por unas medidas y lleva asociada una respuesta; las medidas son las *características* y la respuesta es la *clase*.

**Definición 3** (Característica y vector de características). Una *característica* es cada una de las magnitudes que se miden de un ejemplo y de las que puede depender su respuesta; se corresponde con una columna de la base de datos. El *vector de características* de un ejemplo reúne todas ellas,  $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ , y  $d$  es el número de características.

**Definición 4** (Clase). La *clase* o *etiqueta*  $y$  es la respuesta asociada a un ejemplo, la última columna de la base de datos. En clasificación binaria toma dos valores, y escribimos  $y \in \{0, 1\}$ .

En nuestro caso  $x$  recoge medidas de la relación entre una respuesta y su fuente, e  $y$  vale 1 si la respuesta contiene una alucinación y 0 si no. Con  $n$  ejemplos, la base de datos es el conjunto  $\{(x_i, y_i)\}_{i=1}^n$ , que apiladas por filas forman la matriz de características  $X \in \mathbb{R}^{n \times d}$  y el vector de etiquetas  $y \in \{0, 1\}^n$ . El objetivo es estimar una función  $f: \mathbb{R}^d \rightarrow \{0, 1\}$  que, dado un  $x$  nuevo, prediga su clase. La mayoría de clasificadores binarios no devuelven la clase directamente, sino una probabilidad, la respuesta alucina con probabilidad 0,82; un umbral convierte esa probabilidad en una decisión, y cuanto más caro sea equivocarse, más exigente se pone.

Ajustar el modelo consiste en fijar sus parámetros con un subconjunto de *entrenamiento*. Además de esos parámetros, cada método tiene *hiperparámetros*: opciones que no se aprenden de los datos y que gobiernan su comportamiento, como la profundidad de un árbol o el número de vecinos. El modelo se juzga por cómo predice sobre datos reservados que no ha visto, el conjunto de *test*: lo que importa es **generalizar**, acertar sobre lo nuevo más allá de los ejemplos aprendidos.

El obstáculo para generalizar es el **sobreajuste**: un modelo con demasiada capacidad memoriza el ruido del entrenamiento, se ajusta a sus casualidades y falla fuera de él. El fenómeno se entiende por el compromiso **sesgo–varianza** [13]. El *sesgo* mide cuánto se aleja el modelo de la realidad por ser demasiado rígido: uno muy simple no captura la estructura de los datos y erra sistemáticamente, tanto en entrenamiento como en test. La *varianza* mide cuánto cambia el modelo al cambiar el conjunto de entrenamiento: uno muy flexible se pliega a cada fluctuación y es inestable, brillante en entrenamiento y pobre en test. Reducir uno suele aumentar el otro, y el buen rendimiento está en el punto intermedio, al

que se llega ajustando los hiperparámetros y comprobándolo con las técnicas de validación descritas en la metodología.

Ningún modelo es el mejor para todo problema [25]. La forma de la frontera que separa las clases, el número de características o el tamaño de los datos hacen que unos métodos rindan mejor que otros según el caso: un clasificador lineal fracasa si las clases no se pueden separar por una recta, mientras que uno muy flexible sobreajusta con pocos datos. Por eso en la práctica se entrenan varios y se compara su rendimiento, que es justo lo que hace el capítulo de resultados.

### 1.3.2. Regresión logística

La regresión logística [9] es el clasificador lineal por excelencia y sirve de **modelo base**: la referencia simple e interpretable contra la que se miden los demás. Pese a su nombre, resuelve una clasificación, no una regresión. La idea es modelar la probabilidad de la clase positiva como una combinación lineal de las características, pero acotada al intervalo  $(0, 1)$  para que pueda leerse como probabilidad. El acotado lo hace la función *sigmoide*:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^\top x, \quad p(y = 1 \mid x) = \sigma(z). \quad (1.7)$$

Aquí  $\beta_0 \in \mathbb{R}$  es el término independiente y  $\beta \in \mathbb{R}^d$  el vector de coeficientes, uno por característica. La sigmoide (Figura 1.8) comprime cualquier valor real a  $(0, 1)$ : valores muy negativos de  $z$  dan probabilidad cercana a 0, muy positivos cercana a 1, y  $z = 0$  da exactamente 0,5, que marca la frontera de decisión.

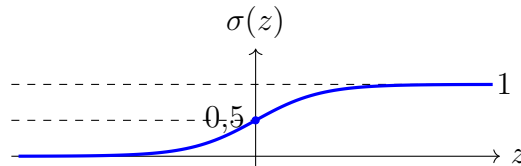


Figura 1.8: Función sigmoide: transforma la combinación lineal  $z$  en una probabilidad entre 0 y 1.

La sigmoide admite una lectura muy útil si se despeja. Partiendo de  $p = \sigma(z) = 1/(1 + e^{-z})$  se tiene  $1 - p = e^{-z}/(1 + e^{-z})$ , y dividiendo,

$$\frac{p}{1 - p} = e^z \implies \log \frac{p}{1 - p} = z = \beta_0 + \beta^\top x. \quad (1.8)$$

La combinación lineal es el logaritmo de la razón entre la probabilidad de la clase positiva y la de la negativa, el *log-odds*. Esta forma da a los coeficientes una lectura directa, y es la mayor virtud del modelo: cada  $\beta_i$  es cuánto varía el log-odds de la clase positiva al aumentar una unidad la característica  $i$ , con las demás fijas. Su signo indica el sentido (un  $\beta_i$  positivo empuja hacia la clase positiva) y su magnitud, la fuerza. Un coeficiente negativo grande sobre el containment, por ejemplo, significaría que a mayor solape con la fuente, menor probabilidad de alucinación, justo lo esperable.

Reducido a una sola característica, el containment  $x$ , el modelo es  $p(x) = \sigma(\beta_0 + \beta_1 x)$ . Un ejemplo fija la lectura. Supongamos  $\beta_0 = 2$  y  $\beta_1 = -4$ , coeficientes que un ajuste podría producir para este problema. Una respuesta con containment  $x = 0,25$  recibe  $z = 2 - 4 \cdot 0,25 = 1$ , luego probabilidad de alucinar  $\sigma(1) \approx 0,73$ : poco solape con la fuente,

sospecha alta. Otra con  $x = 0,75$  recibe  $z = 2 - 4 \cdot 0,75 = -1$  y  $\sigma(-1) \approx 0,27$ : mucho solape, sospecha baja. El signo negativo de  $\beta_1$  expresa justamente lo esperable, a más apoyo en la fuente, menos probabilidad de invención, y la frontera de decisión cae donde  $z = 0$ , esto es, en  $x = -\beta_0/\beta_1 = 0,5$ . Una respuesta nueva se clasifica como alucinación si su probabilidad supera el umbral fijado.

Queda estimar los coeficientes  $\beta$  a partir del entrenamiento. El criterio es la **máxima verosimilitud**: elegir los  $\beta$  que hacen más probables las etiquetas observadas. Con las predicciones abreviadas  $p_i = \sigma(\beta_0 + \beta^\top x_i)$  y suponiendo los ejemplos independientes, la verosimilitud de la base de datos es el producto de las probabilidades que el modelo asigna a cada etiqueta real,

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}, \quad (1.9)$$

donde el exponente selecciona el factor que toca:  $p_i$  si  $y_i = 1$  y  $1 - p_i$  si  $y_i = 0$ . Maximizar un producto de muchos factores es incómodo, así que tomamos logaritmos, que convierten el producto en suma y no mueven el máximo por ser el logaritmo creciente. Cambiando el signo para pasar de maximizar a minimizar y dividiendo por  $n$  para promediar, se obtiene la *entropía cruzada*, la función de pérdida que castiga predecir con confianza la clase equivocada:

$$\mathcal{L}(\beta) = -\frac{1}{n} \log L(\beta) = -\frac{1}{n} \sum_{i=1}^n \left[ y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (1.10)$$

Cuando la etiqueta real es  $y_i = 1$ , solo pesa el término  $-\log p_i$ , que se dispara si el modelo asigna a ese ejemplo una probabilidad  $p_i$  próxima a cero; cuando es  $y_i = 0$ , pesa  $-\log(1-p_i)$ , con el castigo simétrico. A diferencia de la regresión lineal, cuya pérdida cuadrática tiene la solución cerrada  $\hat{\beta} = (X^\top X)^{-1} X^\top y$ , la entropía cruzada no se puede despejar: su gradiente es no lineal en  $\beta$ . Sí es convexa, con lo que tiene un único mínimo, y ese mínimo se alcanza donde el gradiente se anula. Usando la matriz de características  $X \in \mathbb{R}^{n \times d}$ , el vector de etiquetas  $y$  y el vector de predicciones  $p = (p_1, \dots, p_n)^\top$ , el gradiente tiene una forma compacta,

$$\nabla_\beta \mathcal{L}(\beta) = \frac{1}{n} X^\top (p - y), \quad (1.11)$$

el promedio de los errores  $p_i - y_i$  ponderado por las características. Como no se anula en forma cerrada, se resuelve por **descenso de gradiente** [5]: se parte de unos coeficientes cualesquiera y se corrigen paso a paso en la dirección contraria al gradiente,  $\beta \leftarrow \beta - \eta \nabla_\beta \mathcal{L}$ , con  $\eta > 0$  el tamaño de paso, hasta converger. Es el mismo método que está en la base de muchos algoritmos de aprendizaje. Para frenar el sobreajuste se añade un término de **regularización** que penaliza coeficientes grandes: la penalización L2 los encoge de forma suave, y la L1 lleva algunos a cero exacto, con lo que además descarta variables.

- **Ventajas:** interpretable por sus coeficientes, muy barata de entrenar y con probabilidades bien calibradas.
- **Desventajas:** solo traza fronteras lineales, así que se queda corta cuando la separación entre clases es curva; conviene además quitar antes las variables muy correlacionadas entre sí.

### 1.3.3. K vecinos más cercanos

El clasificador de  $k$  vecinos más cercanos (KNN) [8] no construye ningún modelo explícito: guarda los datos de entrenamiento y clasifica cada ejemplo nuevo por **vecindad**,

mirando a qué clase pertenecen los que tiene alrededor. La idea plantea dos preguntas: dónde se sitúan los datos y cómo se mide la cercanía. Lo primero se resuelve con el vector de características, que ubica cada ejemplo como un punto de  $\mathbb{R}^d$ . Para lo segundo se usa una distancia; las más habituales son la euclídea (la del teorema de Pitágoras) y la de Manhattan (la suma de diferencias en valor absoluto, como recorrer una cuadrícula de calles),

$$d_{\text{euclídea}}(p, q) = \sqrt{\sum_{i=1}^d (p_i - q_i)^2}, \quad d_{\text{Manhattan}}(p, q) = \sum_{i=1}^d |p_i - q_i|. \quad (1.12)$$

Ambas son casos de la distancia de Minkowski  $(\sum_i |p_i - q_i|^r)^{1/r}$ , con  $r = 2$  y  $r = 1$  respectivamente: la euclídea mide la distancia en línea recta entre dos puntos, la de Manhattan la que se recorre avanzando por ejes, como quien cruza una ciudad de calles en cuadrícula. Para clasificar un punto nuevo se calculan sus distancias a todos los ejemplos de entrenamiento, se toman los  $k$  más cercanos y se le asigna la clase mayoritaria entre ellos (Figura 1.9). La elección de  $k$  cambia el resultado: en la figura, entre los tres vecinos más próximos gana una clase por dos votos a uno, así que con  $k = 3$  el punto se clasifica en ella; al ampliar el radio a  $k = 5$  entran vecinos más lejanos de la otra clase que pueden invertir el voto. El valor de  $k$  es, por tanto, un hiperparámetro que hay que fijar con cuidado.

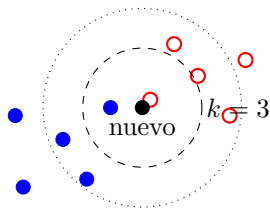


Figura 1.9: KNN: el punto nuevo (negro) se clasifica por la clase mayoritaria entre sus vecinos. El círculo interior es  $k = 3$ ; ampliar el radio ( $k = 5$ ) puede cambiar la decisión.

El parámetro  $k$  gobierna el compromiso sesgo-varianza. Un  $k$  pequeño hace la frontera muy sensible al ruido local: con  $k = 1$  cada punto manda sobre su vecindad inmediata y basta un ejemplo mal etiquetado para crear un islote de clase equivocada, lo que sobreajusta. Un  $k$  grande promedia sobre vecinos cada vez más lejanos, suaviza la frontera y aumenta el sesgo, hasta el extremo de que con  $k = N$  todo punto recibe la clase mayoritaria global. El mejor valor depende de los datos y se busca por validación, aunque una regla común de partida es  $k = \sqrt{N}$ , con  $N$  el número de ejemplos. Como todo se reduce a distancias, KNN es sensible a la escala de las variables y exige estandarizarlas antes. Sin ese paso, una característica medida en miles (por ejemplo, la longitud de la respuesta en caracteres) dominaría la distancia y ahogarían a otra acotada entre 0 y 1 (un solape), que apenas contaría por pequeña que fuese su diferencia; estandarizar deja a todas en el mismo rango y con el mismo peso a priori.

- **Ventajas:** no asume ninguna forma para la frontera entre clases, funciona bien con muchos datos y no requiere una fase de entrenamiento.
- **Desventajas:** clasificar es costoso, pues mide la distancia del punto nuevo a todo el entrenamiento, y el método se degrada cuando hay muchas dimensiones o la distancia está mal elegida.

### 1.3.4. Árboles de decisión

Un árbol de decisión [4, 20] clasifica imitando la forma en que una persona decide a base de preguntas encadenadas, cada una más fina que la anterior, y es además la pieza sobre la que se construyen los dos modelos siguientes. Tiene dos elementos: los *nodos* internos, donde se toma una decisión partiendo los datos por un umbral sobre una característica (“¿containment < 0,5?”), y las *hojas*, que asignan una clase (Figura 1.10). Clasificar un ejemplo consiste en recorrer el árbol desde la raíz, respondiendo en cada nodo la pregunta con los valores de ese ejemplo y siguiendo la rama correspondiente, hasta caer en una hoja, que da la clase predicha.

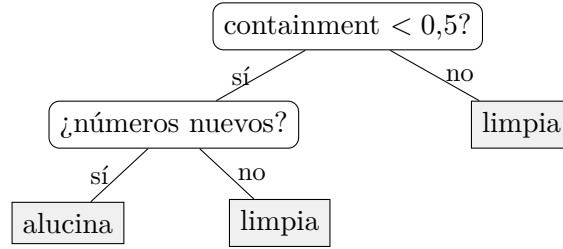


Figura 1.10: Árbol de decisión: los nodos parten los datos por un umbral; las hojas asignan la clase.

El árbol se construye de arriba abajo y de forma recursiva. La pregunta clave en cada nodo es qué corte elegir de entre todos los posibles, y el criterio es la *pureza*: un buen corte deja a cada lado ejemplos lo más de una sola clase que se pueda, porque un grupo homogéneo se etiqueta sin dudar, mientras que uno mezclado deja la decisión en el aire. Para formalizarlo se necesita medir cuánto se mezclan las clases en un nodo, y esa medida es la *impureza*, que se cuantifica con el índice de Gini o con la entropía.

**Definición 5** (Impureza de un nodo). Sea un nodo con proporciones de clase  $p_1, \dots, p_c$  (con  $\sum_k p_k = 1$ ). Su *índice de Gini* y su *entropía* [22] son

$$G = 1 - \sum_{k=1}^c p_k^2, \quad H = - \sum_{k=1}^c p_k \log_2 p_k.$$

Ambas valen cero cuando el nodo es puro (una sola clase, alguna  $p_k = 1$ ) y son máximas cuando las clases están repartidas por igual.

La calidad de un corte se mide por cuánto baja la impureza al pasar del nodo padre a sus dos hijos. Si un nodo padre con impureza  $I(\text{padre})$  se parte en un hijo izquierdo y uno derecho, que reciben una fracción  $w_{\text{izq}}$  y  $w_{\text{der}}$  de los ejemplos, la *reducción de impureza* del corte es

$$\Delta I = I(\text{padre}) - (w_{\text{izq}} I(\text{izq}) + w_{\text{der}} I(\text{der})), \quad (1.13)$$

la impureza del padre menos la media de la de los hijos, ponderada por cuántos ejemplos cae a cada lado. El árbol elige en cada nodo el corte que maximiza  $\Delta I$ . Por ejemplo, un nodo con 40 ejemplos que alucinan y 60 limpios tiene  $G = 1 - 0,4^2 - 0,6^2 = 0,48$ ; un corte que lo separe en un hijo de 30 alucina y 10 limpia ( $G = 0,375$ ) y otro de 10 alucina y 50 limpia ( $G \approx 0,28$ ) deja una impureza media de  $\frac{40}{100} \cdot 0,375 + \frac{60}{100} \cdot 0,28 \approx 0,32$ , con lo que  $\Delta G \approx 0,16$ ; el árbol prefiere ese corte a otro que reduzca menos la impureza. El proceso se repite en cada nodo y se detiene cuando el nodo es puro, no quedan características útiles o se alcanza un límite fijado de antemano.

Sin freno, el árbol crece hasta separar por completo el entrenamiento, memorizándolo, lo que lo hace muy flexible pero propenso al sobreajuste: un árbol profundo aprende las casualidades de los datos y falla con datos nuevos. La profundidad máxima y el número mínimo de ejemplos por hoja son los hiperparámetros que controlan ese crecimiento.

- **Ventajas:** captura interacciones entre variables, es insensible a la escala, maneja bien características irrelevantes y su lógica es legible como una cadena de reglas.
- **Desventajas:** un árbol único es inestable, pues pequeños cambios en los datos dan árboles muy distintos, y sobreajusta con facilidad. Los dos modelos siguientes corrigen esto combinando muchos árboles.

### 1.3.5. Random Forest

Un bosque aleatorio (*Random Forest*) [3] combina muchos árboles para ganar la estabilidad que a uno solo le falta, con la misma lógica por la que la media de muchas opiniones independientes acierta más que una sola. Se apoya en una técnica llamada *bagging* (*bootstrap aggregating*): de la base de entrenamiento se extraen  $B$  muestras aleatorias con reemplazo, cada una del mismo tamaño que la original pero con ejemplos repetidos y otros ausentes, y con cada muestra se entrena un árbol distinto. Para clasificar un ejemplo nuevo, cada uno de los  $B$  árboles emite su voto y se toma la clase más votada,

$$\hat{y}(x) = \text{clase más votada entre } \{T_1(x), \dots, T_B(x)\}. \quad (1.14)$$

El bagging reduce la varianza sin aumentar el sesgo, y merece la pena ver por qué con un poco de detalle. Pensemos primero en la versión con predicciones numéricas, donde cada árbol devuelve la probabilidad de la clase positiva y el conjunto las promedia. Si cada árbol individual tiene varianza  $\sigma^2$  y dos árboles distintos tienen una correlación  $\rho$  entre sus predicciones, la varianza del promedio de los  $B$  árboles es

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b(x)\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2. \quad (1.15)$$

El segundo sumando se desvanece al crecer  $B$ , así que promediar muchos árboles cancela la parte no correlacionada del ruido, igual que promediar muchas mediciones ruidosas se acerca al valor real. El primer sumando, en cambio, no depende de  $B$ : queda un suelo de varianza  $\rho \sigma^2$  que solo baja si los árboles están poco correlacionados. Como el promedio no cambia la predicción esperada, el sesgo se mantiene. De aquí sale la receta del bosque: para bajar ese suelo, hay que des-correlacionar los árboles. El bagging ya los diferencia al entrenar cada uno con una muestra distinta, y el bosque añade un segundo golpe de azar: en cada corte no considera todas las características, sino un subconjunto aleatorio de ellas (típicamente  $\sqrt{d}$  de las  $d$  disponibles en clasificación). Así ningún árbol se apoya siempre en la misma variable dominante, baja  $\rho$  y el conjunto gana estabilidad. El número de árboles  $B$ , su profundidad y cuántas características se consideran en cada corte son sus hiperparámetros, y suelen buscarse probando.

- **Ventajas:** robusto y preciso, uno de los mejores clasificadores para datos tabulares; hereda del árbol la captura de interacciones y la insensibilidad a la escala, maneja muchas variables y apenas sobreajusta.
- **Desventajas:** pierde la interpretabilidad de un árbol único, es más costoso de entrenar y evaluar, y puede desajustarse con datos muy ruidosos si no se controla el número de árboles.

### 1.3.6. Gradient boosting

El *gradient boosting* [12] también combina árboles, pero de forma **secuencial** en vez de paralela. En lugar de promediar árboles independientes que votan a la vez, construye el modelo por etapas, al modo de un estudiante que resuelve un problema, mira en qué se equivocó y dedica el siguiente intento a corregir justo esos fallos. Empieza con una predicción sencilla y, en cada etapa, añade un árbol nuevo entrenado para reducir el error que dejan los anteriores. Ese error a corregir es el gradiente de la función de pérdida evaluado en las predicciones actuales, de ahí el nombre del método. El modelo tras  $m$  etapas es la suma del anterior más el árbol nuevo,

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \quad (1.16)$$

donde  $h_m$  es el árbol de la etapa  $m$ , ajustado a los gradientes que deja  $F_{m-1}$ , y  $\nu \in (0, 1]$  es la *tasa de aprendizaje*, que modera cuánto contribuye cada árbol. Una tasa pequeña avanza despacio pero generaliza mejor; una grande, o demasiados árboles, sobreajustan, así que tasa y número de árboles se ajustan juntos.

La diferencia con el bosque aleatorio es de fondo. El bosque promedia árboles independientes para reducir la *varianza*, partiendo de árboles que ya de por sí aciertan; el boosting encadena árboles dependientes, cada uno atacando lo que el anterior no supo, para reducir el *sesgo*, partiendo de un modelo débil que va mejorando. Eso suele dar mayor precisión, a cambio de un ajuste más delicado y más lento. **XGBoost** [7] es una implementación eficiente y regularizada de esta idea, referencia habitual en problemas sobre datos tabulares y uno de los cinco clasificadores que compararemos.

- **Ventajas:** suele ofrecer la mayor precisión de los modelos clásicos sobre datos tabulares y maneja bien las interacciones entre variables.
- **Desventajas:** tiene más hiperparámetros que ajustar, entrena más despacio que un bosque y su riesgo de sobreajuste es mayor si no se regula con cuidado.



# Bibliografía

- [1] Charu C. Aggarwal and ChengXiang Zhai. *Mining Text Data*. Springer, New York, 2012.
- [2] Ana Azevedo and Manuel Filipe Santos. KDD, SEMMA and CRISP-DM: A parallel overview. In *Proceedings of the IADIS European Conference on Data Mining*, pages 182–185, Amsterdam, 2008.
- [3] Leo Breiman. Random forests. *Machine Learning*, 45(1), 2001.
- [4] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, Monterey, CA, 1984.
- [5] Augustin-Louis Cauchy. Méthode générale pour la résolution des systèmes d’équations simultanées. *Comptes Rendus Hebdomadaires des Séances de l’Académie des Sciences*, 25:536–538, 1847.
- [6] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 2002.
- [7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [8] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 1967.
- [9] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2), 1958.
- [10] Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6), 1990.
- [11] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 2006.
- [12] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 2001.
- [13] Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992.

- [14] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 2003.
- [15] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3), 2002.
- [16] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2), 2011.
- [17] James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- [18] Zellig S. Harris. Distributional structure. *WORD*, 10(2–3):146–162, 1954.
- [19] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.
- [20] J. Ross Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986.
- [21] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 1988.
- [22] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948.
- [23] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–147, 1974.
- [24] Rüdiger Wirth and Jochen Hipp. CRISP-DM: Towards a standard process model for data mining. In *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining (PADD)*, pages 29–40, Manchester, 2000.
- [25] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996.
- [26] William J. Youden. Index for rating diagnostic tests. *Cancer*, 3(1):32–35, 1950.